DRAFT: Mechanizing Choreographies with State Transformers

Timon Böhler
Technische Universität Darmstadt, Germany

Simon Daniel
Technische Universität Darmstadt, Germany

David Richter
Technische Universität Darmstadt, Germany

Pascal Weisenburger
University of St. Gallen, Switzerland

Mira Mezini
Technische Universität Darmstadt, Germany; hessian.AI; ATHENE-Center

Abstract

Choreographic programming is a programming model for developing distributed applications where an entire communication protocol is written as a single program, which a compiler then projects to one process per participant. Choreographic programming abstracts over low-level network communication primitives such as sockets, and provides a high degree of safety guarantees with deadlock freedom ensured by construction.

Mechanizing choreographies necessarily deals with both operations specific to distributed programming and standard (local) operations that also occur in non-distributed programs, as well as the typical issues of binding and substitution. We aim to sidestep the latter issues, thereby obtaining a more concise mechanization that focuses on the essential distributed aspects of choreographies.

To this end, we use a method recently proposed by Thiemann to elegantly model deadlock-free processes in a dependently typed language: Using state transformers to represent the computations performed by each process. We bring the state transformer model to choreographies, allowing us to reducing the usual mechanization effort around binding and substitution, and to abstract over the details of the "local" aspects of the language.

We mechanize in Lean a choreographic language that supports point-to-point communication, broadcasting, recursive procedures, and *local stateful methods*, allowing each participant to be assigned a different set of methods. We prove soundness and completeness of endpoint projection and establish deadlock freedom for the projected processes.

2012 ACM Subject Classification Replace ccsdesc macro with valid one

Keywords and phrases Dummy keyword

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Supplementary Material *Software (Lean source code)*: `TODO`

Funding *Timon Böhler*: TODO FUNDING
Simon Daniel: EmergenCITY
David Richter: TODO FUNDING
Pascal Weisenburger: TODO FUNDING
Mira Mezini: TODO FUNDING

1 Introduction

Distributed programs are notoriously hard to write. Two major reasons are the complexity of managing communication between multiple programs and the risk of deadlocks. Choreo-

© Timon Böhler, Simon Daniel, David Richter, Pascal Weisenburger, and Mira Mezini;
licensed under Creative Commons License CC-BY 4.0
42nd Conference on Very Important Topics (CVIT 2016).
Editors: John Q. Open and Joan R. Access; Article No. 23; pp. 23:1–23:18
Leibniz International Proceedings in Informatics
Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

graphic programming [18] promises to alleviate these issues by allowing distributed programs to be described using a *global* view where the behavior of all participants is contained in a single program. To execute choreographies, they are *projected* to a set of communicating processes (or *endpoints*). The main correctness properties one wants to ensure are that (i) endpoint projection preserves the semantics of the choreography and (ii) the projected network is free from deadlocks.

While many choreographic languages with wildly differing feature sets have been proposed, *mechanizations* of choreographies are rarer, and are often quite complex, relying on intricate proof developments [10]. For instance, a recent mechanization of multiparty session types [6] notes that for the most difficult theorem, "the majority of the proof is about dealing with binders, renaming, etc.". In this work, we demonstrate an encoding of choreographies using *state transformers*, which have previously been applied by Thiemann [22] to the modeling of processes in the context of session types. The idea is that each process is assigned a state (of an arbitrary type). Operations like sending and receiving are then modelled by meta-language functions which read the state to produce a message or which read a message and return a new state. This encoding has the advantage that it does not require a context of variables for each participant, avoids issues relating to naming and binding, and does not require explicitly defining a notion of locally evaluated expressions, as the embedding automatically supports arbitrary meta-language expressions. We therefore get a neat separation between the essential, distributed features of choreographies and the local aspects that do not influence the proof of correctness. Additionally, embedded programs using state transformers tend to be more readable than approaches using encodings like de Bruijn indices or locally nameless.

Using this encoding, we provide a mechanized proof of the soundness and completeness of endpoint projection and of deadlock freedom of projected networks. The choreographies we describe can (i) send messages between two participants, (ii) switch between branches of the choreography by communicating to each participant a message stating which branch was chosen, (iii) call (potentially recursive) procedures, and (iv) perform role-dependent local operations. The last feature is formalized by mapping a set of state-transformer functions to each role, allowing the set of methods to differ between different participants (for example, a participant might have a method for modifying a local database). To our knowledge, this feature has not previously been formalized, and the ease of mechanizing it underscores the benefits of the state-transformer approach.

Our contributions are:

- A formalization of choreographies via state transformers, which avoids tedious handling of naming and variable binding.
- A mechanized proof of soundness and completeness of endpoint projection, and of deadlock freedom for projected networks.
- The introduction and mechanization of role-dependent methods.

The paper is structured as follows: In Section 2, we introduce choreographies, go through the set of features our choreographic calculus supports, and explain the embedding using state transformers. In Section 3 describes the formalization of choreographies and processes, and of their small-step operational semantics and of projection. More importantly, it contains the soundness, completeness and deadlock freedom proofs, and the proof of partial correctness for an interpreter with respect to the operational semantics. We discuss related work in Section 4 before concluding in Section 5.

Colors Following Patrignani's [19] recommendation to visually distinguish the syntax of different languages in the same paper, we distinguish choreographies and processes through

different colors and different fonts: Choreographic syntax is in a blue sans-serif font (e.g.: done), while process syntax is in a red bold serif font (e.g.: **done**).

2 Representing Processes and Choreographies

2.1 State Transformers

When participants in a choreography receive messages or call local methods, their state changes. Standard approaches model the receiving of messages by extending the participant's environment, i.e. by binding a new variable. In the state transformer approach, in contrast, the participant's state is a value of an arbitrary type (which can be different between each participant).

Communication in our language is formalized as follows.

$$\frac{p, q : \mathsf{Role} \qquad p \neq q \qquad c : \mathsf{Chor} \qquad v : \mathsf{LState}_p \to m \qquad u : \mathsf{LState}_q \times m \to \mathsf{LState}_q}{\mathsf{com}_{p,q}(v, u); c : \mathsf{Chor}}$$

A communication from role `p` to role `q` is defined by two functions: `v` describes how `p` computes the message that it sends from its own state, while `u` describes how `q` updates its own state upon receiving the message. Note that the message type `m` is a parameter of the `com` constructor, meaning that each occurrence of `com` in a program can have a different message type.

We describe a simple protocol between two participants alice and bob, in which alice tries to guess bob's secret number, upon which he tells her whether she guessed correctly. Each participant's state is a First, alice sends a message containing her guess; upon receiving the message, bob compares it to his secret and tells alice about the result.

$$\mathsf{com}_{\mathsf{alice},\mathsf{bob}}((\lambda_.\ \mathrm{guess}), (\lambda s\ g.\ g = \mathrm{secret}));\qquad \mathsf{com}_{\mathsf{bob},\mathsf{alice}}((\lambda b.\ b), (\lambda s\ b.\ b));\qquad \mathsf{done}$$

We show the Lean code below. The argument `(by decide)` is needed because our formalization assumes that the sender and receiver of a communication are distinct, and hence a proof must be provided.

```
.com .alice .bob (by decide) (λ _ => guess) (λ _ m => m == secret) <|
.com .bob .alice (by decide) (λ b => b) (λ _ b => b) <|
.done
```

Simulating variables. As mentioned, we use state transformers instead of variable contexts. We now show how variable context can be simulated in our approach. In addition, it is easy to see that no process can read another process' data, as the functions all operate on the local state only. When using normal contexts, one has to explicitly keep track which variables are accessible from which process.

Variables can be modeled by instantiating each process' local state with its variable context. Consider the following example where the process p's state has type $\{a : \mathbb{N}, b : \mathbb{N}\}$ and the process q's state has type $\{x : \mathbb{N}, y : \mathbb{N}\}$. The choreography first has p send $a + 1$ to q, which q stores in x. Then q sends $x + y$ to p, which p stores in b. On the left, we write the program in our representation; on the right, we show pseudocode (inspired by HasChor [20]) for comparison.

$$\begin{array}{l}\mathsf{com}_{p,q}((\lambda s.\ s.a+1),(\lambda s\ x'.\ s[x \mapsto x']));\\ \mathsf{com}_{q,p}((\lambda s.\ s.x+s.y),(\lambda s\ b'.\ s[b \mapsto b']));\\ \mathsf{done}\end{array}$$

```
let x ← (p, a + 1) ~> q
let b ← (q, x + y) ~> p
return ()
```

While most choreographic languages use a functional style where messages introduce new variable bindings, our representation using state transformers is also very similar to the language used by Cruz-Filipe and Montesi [8] (but not mechanized) to write *imperative* algorithms as choreographies.

2.2 Broadcasting

Our choreographic language supports branching. For branching to be projectable to a network of processes, it is necessary that every involved process knows which branch has been taken. We follow HasChor's approach in adding a $\mathsf{bcast}_p(v); c$ construct, in which the process p transmits the same message to all participants (computed by v), which is then used to choose which branch to take. Here, c is a continuation which takes the message's value and returns a choreography (i.e., syntax). The idea of representing the continuation of an operation as a meta-language from values to syntax is widely used in embedding effectful languages [7], like in Freer Monads [16] or Interaction trees [24]. Broadcasting can hence be seen as a globally executed effect whose result determines how to choreography continues.

We demonstrate broadcasting below for an example where the message type is Boolean, meaning we choose between two branches. Again, Alice tries to guess Bob's secret, but this time the result decides whether Bob sends Alice a message in return.

$$\begin{array}{l}\mathsf{com}_{\mathsf{alice},\mathsf{bob}}((\lambda_.\ \text{guess}),(\lambda_.\ \lambda m.\ m = \text{secret}));\\ \mathsf{bcast}_{\mathsf{bob}}((\lambda s.\ s));\\ \lambda m.\ \begin{cases}\mathsf{com}_{\mathsf{bob},\mathsf{alice}}((\lambda_.\ \text{"hello"}),(\lambda s_.\ s));\mathsf{done} & \text{if } m = \mathsf{true}\\ \mathsf{done} & \text{if } m = \mathsf{false}\end{cases}\end{array}$$

In Lean, the example looks as follows:

```
.com .alice .bob (by decide) (λ _ => guess) (λ _ m => m == secret) <|
.bcast .bob (λ s => s) λ
  | true => .com .bob .alice (by decide) (λ _ => "hello") (λ s _ => s) .done
  | false => .done
```

2.3 Methods

In a distributed network, different participants may have different capacities, for example, one participant may have access to a database that others do not have access to. We model this using role-dependent methods. More precisely, our approach allows developers to define a type family `Method` mapping each role to a type representing the role's methods. They then define a function `I` which maps each method to its interpretation, which is a function that takes the role's state and returns a new state.

In the example below, the participant Bob has access to a method `randNat` (implemented using the Lean function of the same name), allowing him to generate random numbers. His state consists of the most recently generated number and the state used by the number generator. The choreography consists of Bob generating a random number and passing it to Alice, who adds it to her current state.

$$\mathsf{run}_{\mathsf{bob}}(\mathsf{randNat}(1,100));\qquad \mathsf{com}_{\mathsf{bob,alice}}((\lambda(n,_.\ n),(\lambda s.\ \lambda n.\ s+n));\qquad \mathsf{done}$$

To define the choreography in Lean, we first define Bob's type of methods `MethodBob`. We then define the choreography's signature (more on that in Section 3), which means specifying the state type for each process, as well as the available methods and there interpretation.

```
inductive MethodBob where | randNat (lo hi : Nat)

instance RandEx : Sig where -- unused parameters omitted
  LState | .alice => Nat    | .bob => Nat × StdGen
  Method | .alice => Empty | .bob => Method
  I      | .bob, .randNat lo hi, (_, gen) => randNat gen lo hi

def randex : @Chor RandEx :=
  .run .bob (.randNat 1 100) <|
  .com .bob .alice (by decide) (λ (n, _) => n) (λ (s n : Nat) => s + n) .done
```

Procedures Similar to methods are procedures, which however evaluate to choreographies, can be recursive, and are called globally. The syntax is $\mathsf{call}(f);c$, where f is a procedure name.

2.4 Interpretation

Another advantage of the use of state transformers and mixed embeddings is how easily a toy interpreter can be written. The interpreter below takes a set of procedures, a global state, and a choreography. It evaluates the choreography (deterministically) and outputs the final global state. Here, $\mathcal{D}$ is a function mapping each procedure to its function body, g is the contains each processes state, $\fatsemi$ sequentially composes two choreographies, $g[p \mapsto x]$ sets the state of process p to x, and $\mathsf{I}_p(e, g_p)$ runs method e on the local state of process p.

$$\begin{aligned}
\mathcal{I}(\mathcal{D}, g, \mathsf{run}_p(e); c) &\triangleq \mathcal{I}(d, g[p \mapsto \mathsf{I}_p(e, g_p)], c)\\
\mathcal{I}(\mathcal{D}, g, \mathsf{com}_{p,q}(v, u); c) &\triangleq \mathcal{I}(d, g[q \mapsto u(g_q, v(g_p))], c)\\
\mathcal{I}(\mathcal{D}, g, \mathsf{bcast}_p(v); c) &\triangleq \mathcal{I}(d, g, c(v(g_p)))\\
\mathcal{I}(\mathcal{D}, g, \mathsf{call}(f); c) &\triangleq \mathcal{I}(d, g, \mathcal{D}(f) \fatsemi c)\\
\mathcal{I}(\mathcal{D}, g, \mathsf{done}) &\triangleq g
\end{aligned}$$

Note that the definition is not structurally recursive. The equations still have a solution, as the function is tail recursive. This is demonstrated by the Lean implementation, which passes the type checker by using the `partial_fixpoint` keyword.

```
def Chor.interpP (d : sig.Func → Chor) (g : GState) : Chor → GState
| run p e c => c.interpP d g[p ↦ sig.I p e (g p)]
| com p q _h v u c => c.interpP d g[q ↦ u (g q) (v (g p))]
| bcast p v c => (c (v (g p))).interpP d g
| call x c => ((d x).seq c).interpP d g
| done => g
partial_fixpoint
```

We have further implemented a version of the interpreter using a fuel argument and the `Option` monad (see Section 3.6), for which we have proven partial correctness, i.e. if it returns some final state, that state is reachable in the operational semantics. All our example programs also work with the verified fuel-based interpreter.

$$\frac{p : \mathsf{Role} \qquad e : \mathsf{Method}(p) \qquad c : \mathsf{Chor}}{\mathsf{run}_p(e); c : \mathsf{Chor}} \qquad \frac{\begin{array}{c} p, q : \mathsf{Role} \qquad p \neq q \qquad c : \mathsf{Chor} \\ v : \mathsf{LState}_p \to m \qquad u : \mathsf{LState}_q \times m \to \mathsf{LState}_q \end{array}}{\mathsf{com}_{p,q}(v, u); c : \mathsf{Chor}}$$

$$\frac{\begin{array}{c} p : \mathsf{Role} \\ v : \mathsf{LState}_p \to m \\ c : m \to \mathsf{Chor} \end{array}}{\mathsf{bcast}_p(v); c : \mathsf{Chor}} \qquad \frac{c : \mathsf{Chor} \qquad f : \mathsf{Func}}{\mathsf{call}(f); c : \mathsf{Chor}} \qquad \frac{}{\mathsf{done} : \mathsf{Chor}}$$

Figure 1 Intrinsically typed syntax of choreographies.

3 Mechanization

In this section, we mechanize choreographies and processes. Our main result is deadlock freedom, which is a central guarantee provided by choreographic programming.

3.1 Signature

Our development is parametrized over (i) a type Role of process identifiers, which is assumed to have decidable equality, (ii) a type LState of local states for each process identifier, (iii) a type Method of methods for each process identifier that are available there locally, (iv) a type Func of function identifiers, and (vi) a function I interpreting each method as a state transformer.

$$\mathsf{Role} : \mathsf{Type} \qquad \mathsf{LState} : \mathsf{Role} \to \mathsf{Type} \qquad \mathsf{Method} : \mathsf{Role} \to \mathsf{Type} \qquad \mathsf{Func} : \mathsf{Type}$$

$$\mathsf{I} : (p : \mathsf{Role}) \to \mathsf{Method}_p \to \mathsf{LState}_p \to \mathsf{LState}_p$$

We additionally require Role to be non-empty (`Inhabited`) and to have decidable equality, as made explicit by the Lean mechanization below, where the parameters are captured in a type class, to enable them being passed implicitly, reducing notational overhead.

```
class Sig where
  (Role : Type) (inhRole : Inhabited Role) (decRole : DecidableEq Role)
  LState : Role → Type
  Method : Role → Type
  Func : Type
  I : (p : Role) → Methods p → LState p → LState p
```

3.2 Syntax

Choreographies. We model choreographies as sequential programs featuring message passing, branching, local methods, and recursive procedures. The syntax and typing rules for choreographies are given in Figure 1. We formalize choreographies using an inductive type Chor, which is parametrized over a signature Method, giving each process a set of methods. Chor is constructed by one of the following rules:

- $\mathsf{run}_p(e); c$ – Run a method e on process p from the process-specific method set Method_p, then continue with c.

$$\frac{e : s \qquad b : \mathsf{Proc}_p}{\mathbf{run}(e); b : \mathsf{Proc}_p} \qquad \frac{q : \mathsf{Role} \quad b : \mathsf{Proc}_p \quad v : \mathsf{LState}_p \to m}{\mathbf{send}_q(v); b : \mathsf{Proc}_p} \qquad \frac{q : \mathsf{Role} \quad b : \mathsf{Proc}_p \quad u : \mathsf{LState}_p \times m \to \mathsf{LState}_p}{\mathbf{recv}_q(u); b : \mathsf{Proc}_p}$$

$$\frac{v : \mathsf{LState}_p \to m \qquad b : m \to \mathsf{Proc}_p}{\mathbf{shout}(v); b : \mathsf{Proc}_p} \qquad \frac{q : \mathsf{Role} \qquad b : m \to \mathsf{Proc}_p}{\mathbf{listen}_q; b : \mathsf{Proc}_p} \qquad \frac{}{\mathbf{done} : \mathsf{Proc}_p}$$

Figure 2 Intrinsically typed syntax of processes.

- $\mathsf{com}_{p,q}(v, u); c$ – Send a message (computed from the message state by $v : \mathsf{LState}_p \to m$) from process p to q (with proof that $p \neq q$). After communicating, process q updates its local state using $u : \mathsf{LState}_q \to m \to \mathsf{LState}$, and the choreography continues as c.
- $\mathsf{bcast}_p(v); c$ – Broadcast from process p, sending a message computed by $v : \mathsf{LState}_p \to m$, with the continuation depending on the message, $c : m \to \mathsf{Chor}$. The broadcast involves all participants, following the approach taken by HasChor [20].
- $\mathsf{call}(f); c$ – Call a procedure f and continue with c.
- done – The terminal choreography, indicating no further behavior.

In Lean, the syntax is defined as an inductive type, parametrized over a signature `sig`.

```
inductive Chor [sig : Sig] : Type 1
| run (p: sig.Role) (eff: sig.Methods p) (c: Chor)
| com {m : Type} (p q : sig.Role) (h : p ≠ q) (v : sig.LState p → m) (u :
    sig.LState q → m → sig.LState q) (c : Chor)
| bcast {m : Type} (p : sig.Role) (v : sig.LState p → m) (c : m → Chor)
| call (x : sig.Func) (c : Chor)
| done
```

Processes. The type Proc_p in Figure 2 models the behavior of individual processes. It captures the local actions a process can perform, including method execution, sending a message, receiving a message, and state-dependent branching. The constructors of Proc_p are:

- $\mathbf{run}(e); b$: Call a method e (of type s) and continue with b.
- $\mathbf{send}_p(v); b$ – Send a message to process p, where the message is computed by $v : \mathsf{LState} \to m$, then continue as b.
- $\mathbf{recv}_p(u); b$ – Receive a message from process p, updating the local state via $u : \mathsf{LState} \times m \to \mathsf{LState}$, then continue as b.
- $\mathbf{shout}(v); b$ – Broadcast a message m computed by v, then feed m to the continuation $b : m \to \mathsf{Proc}_{p,s}$.
- $\mathbf{listen}_p; b$ – Wait for a broadcast from process p, then continue based on the received message with $b : m \to \mathsf{Proc}_p$.
- $\mathbf{call}(f); b$ – Call procedure f, then continue with b.
- $\mathbf{done}$ – Terminate.

The corresponding Lean code is:

```
inductive Proc (p : sig.Role) : Type 1
| eff (e: sig.Method p) (c: Proc p)
| send {m : Type} (q : sig.Role) (v : sig.LState p → m) (b : Proc p)
```

$$
\begin{aligned}
[\![c]\!] &: \mathsf{Chor} \to \mathsf{Netw} \\
[\![\mathsf{run}_p(e); c']\!]_r &\triangleq \begin{cases} \mathbf{run}(e); [\![c']\!]_r & \text{if } r = p \\ [\![c']\!]_r & \text{if } r \neq p \end{cases} \\
[\![\mathsf{com}_{p,q}(v,u); c']\!]_r &\triangleq \begin{cases} \mathbf{send}_q(v); [\![c']\!]_r & \text{if } r = p \\ \mathbf{recv}_p(u); [\![c']\!]_r & \text{if } r = q \\ [\![c']\!]_r & \text{if } r \notin \{p,q\} \end{cases} \\
[\![\mathsf{bcast}_p(v); c']\!]_r &\triangleq \begin{cases} \mathbf{shout}(v); \lambda m.[\![c'(m)]\!]_r & \text{if } r = p \\ \mathbf{listen}_p; \lambda m.[\![c'(m)]\!]_r & \text{if } r \neq p \end{cases} \\
[\![\mathsf{call}(f); c]\!]_r &\triangleq \mathbf{call}(f); [\![c]\!]_r \\
[\![\mathsf{done}]\!]_r &\triangleq \mathbf{done}
\end{aligned}
$$

Figure 3 Projection from choreographies to networks.

```
| recv {m : Type} (q : sig.Role) (u : sig.LState p → m → sig.LState p) (b : Proc
    p)
| broad {m : Type} (v : sig.LState p → m) (b : m → Proc p)
| switch {m : Type} (q : sig.Role) (b : m → Proc p)
| call (x : sig.Func) (b : Proc p)
| done
```

3.3 Endpoint Projection

The endpoint projection function $[\![\cdot]\!]$ maps a choreography to a network, which assigns each process its behavior extracted from the choreography. Networks are defined as follows:

$$\mathsf{Netw} \triangleq (p : \mathsf{Role}) \to \mathsf{Proc_p}.$$

We formally define $[\![\cdot]\!]$ in Figure 3, such that $[\![c]\!]_r$ is the projection of choreography c to process r. When the choreography calls a method at process p, only the projection of p will contain this method, while other processes skip it and continue projecting the continuation. When the choreography communicates a message from p to q, the projection constructs a send action for p, a receive action to q, and other processes proceed with the continuation. When the choreography branches via a broadcast at process p, the projection constructs a shout for p, and a corresponding listen for other processes. When the choreography is done, all processes are done.

3.4 Semantics

In this section, we describe the operational semantics (as labelled transition systems) for choreographies and *networks*, which are assignments of processes to roles.

For both choreographies and networks, a step is labeled with the type of operation that is performed and the processes that perform the step. Concretely, the possible labels are:

$$\mathsf{Label} ::= \mathsf{run}_p \mid \mathsf{com}_{p,q} \mid \mathsf{bcast}_p \mid \mathsf{call}$$

When proving soundness, the label of a network step tells us which processes just performed an operation, allowing us to determine the corresponding choreography step.

The semantics additionally needs to evaluate procedure calls. For this, a parameter $\mathcal{D}$ is assumed, which maps each procedure to its definition.

$$\mathcal{D} : \mathsf{Func} \to \mathsf{Chor}$$

Choreographies. The operational semantics for choreographies is given in Figure 4. It is a binary relation over tuples of the form $\langle c \mid g \rangle$, where c is a choreography and a global state $g : \mathsf{GState}$, which maps process identifiers to their local state:

$$\mathsf{GState} \quad \triangleq \quad (p : \mathsf{Role}) \to \mathsf{LState}_p$$

A method e on process p is run by applying its semantics $I_p(e, g_p)$ and updating p's state accordingly. When executing a communication $\mathsf{com}_{p,q}(v, u)$, we update the state of the receiver (q) using u.

The computation rules, Crun, Ccom, Cbcast, and Ccall, correspond directly to the syntactic constructs of the choreography. Rule Crun executes a method on a specific process p, updating the global state g at p via the interpretation function I_p. Rule Ccom models directed communication; it updates the state of the receiver q using the update function u and the value computed from the sender p's state. Rule Cbcast evaluates a function on the local state of a process p, and then chooses how to continue the rest of the choreography based on the result. Rule Ccall evaluates a call to a procedure x by inserting its body into the choreography. The rule makes use of an operator $\fatsemi$ which sequences two choreographies and which is defined in Figure 5. The operator $\fatsemi$ is defined by recursion on its left argument, in a similar fashion to the standard definition of `append` for linked lists. Note that our semantics makes procedure calls *synchronized*, i.e. all processes are assumed to enter the procedure together. It is possible to model asynchronicity by introducing intermediate states in which only a subset of processes has entered the procedure [18], but this is orthogonal to our use of state transformers, so we chose to take the simpler synchronous version.

The remaining rules, Cdelay1 through Cdelay4, introduce nondeterminism into the semantics. Although the syntax of choreographies is sequential, actions involving disjoint sets of processes are independent when executed on a network. Nondeterminism is hence needed to match the choreography semantics with the network semantics, where nondeterminism is inherent [10]. For example, in the program $\mathsf{run}_p(e); \mathsf{run}_q(f); \mathsf{done}$, if $p \neq q$, then either p or q could execute its method first.

Networks. We define the operational semantics on networks in Figure 6. It is defined on configurations of the form $\langle n \mid g \rangle$, where n is a network and g is a global state. The rules for methods and communication resemble the rules for choreographies, except we now perform steps on arbitrary processes in the network. Note that the rules are directly defined on a network, given as a function from roles to processes. This is in contrast to the approach taken by Cruz-Filipe et al. [10], where networks form a partial commutative monoid, allowing them to be composed in parallel. We find our representation more suitable for theorem proving, as we avoid the known difficulties of partial operations and furthermore do not need a congruence rule as in Cruz-Filipe et al. [10]'s approach.

The rule Nrun advances a single process p by executing a local method and updating the network mapping n so that p proceeds to its continuation b. The rule Ncom models communication: a step occurs only when a sender p (ready to **send**) and a receiver q (ready to **recv**) are both at the head of their respective processes. The rule updates the receiver's state and advances both processes simultaneously. In rule Nbcast, process p must **shout** a

$$\textsc{Crun}\quad \frac{g' = g[p \mapsto I_p(e, g_p)]}{\langle \mathsf{run}_p(e); c \mid g\rangle \rightarrow^{\mathsf{run}_p} \langle c \mid g'\rangle}$$

$$\textsc{Ccom}\quad \frac{g' = g[q \mapsto u(g_q, v(g_p))]}{\langle \mathsf{com}_{p,q}(v, u); c \mid g\rangle \rightarrow^{\mathsf{com}_{p,q}} \langle c \mid g'\rangle}$$

$$\textsc{Cdelay1}\quad \frac{p \neq r \quad p \neq s \quad q \neq r \quad q \neq s \quad \langle c \mid g\rangle \rightarrow^{\mathsf{com}_{r,s}} \langle c' \mid g'\rangle}{\langle \mathsf{com}_{p,q}(v, u); c \mid g\rangle \rightarrow^{\mathsf{com}_{r,s}} \langle \mathsf{com}_{p,q}(v, u); c' \mid g'\rangle}$$

$$\textsc{Cdelay2}\quad \frac{p \neq r \quad \langle c \mid g\rangle \rightarrow^{\mathsf{run}_r} \langle c' \mid g'\rangle}{\langle \mathsf{run}_p(e); c \mid g\rangle \rightarrow^{\mathsf{run}_r} \langle \mathsf{run}_p(e); c' \mid g'\rangle}$$

$$\textsc{Cdelay3}\quad \frac{p \neq r \quad p \neq s \quad \langle c \mid g\rangle \rightarrow^{\mathsf{com}_{r,s}} \langle c' \mid g'\rangle}{\langle \mathsf{run}_p(e); c \mid g\rangle \rightarrow^{\mathsf{com}_{r,s}} \langle \mathsf{run}_p(e); c' \mid g'\rangle}$$

$$\textsc{Cdelay4}\quad \frac{p \neq r \quad q \neq r \quad \langle c \mid g\rangle \rightarrow^{\mathsf{run}_r} \langle c' \mid g'\rangle}{\langle \mathsf{com}_{p,q}(v, u); c \mid g\rangle \rightarrow^{\mathsf{run}_r} \langle \mathsf{com}_{p,q}(v, u); c' \mid g'\rangle}$$

$$\textsc{Cbcast}\quad \frac{}{\langle \mathsf{bcast}_p(v); c \mid g\rangle \rightarrow^{\mathsf{bcast}_p} \langle c(v(g_p)) \mid g\rangle}$$

$$\textsc{Ccall}\quad \frac{}{\langle \mathsf{call}(f); c \mid g\rangle \rightarrow^{\mathsf{call}} \langle \mathcal{D}(f) \fatsemi c \mid g\rangle}$$

Figure 4 Operational Semantics of Choreographies

$$\begin{array}{rcl}
(\cdot \fatsemi \cdot) & : & \mathsf{Chor} \rightarrow \mathsf{Chor} \rightarrow \mathsf{Chor} \\
(\mathsf{run}_p(e); c) \fatsemi c' & \triangleq & \mathsf{run}_p(e); (c \fatsemi c') \\
(\mathsf{com}_{p,q}(v, u); c) \fatsemi c' & \triangleq & \mathsf{com}_{p,q}(v, u); (c \fatsemi c') \\
(\mathsf{bcast}_p(v); c) \fatsemi c' & \triangleq & \mathsf{bcast}_p(v); (\lambda m.\ (c\ m) \fatsemi c') \\
(\mathsf{call}(f); c) \fatsemi c' & \triangleq & \mathsf{call}(f); (c \fatsemi c') \\
\mathsf{done} \fatsemi c' & \triangleq & c'
\end{array}$$

Figure 5 Sequencing of choreographies.

$$\textsc{Nrun}\quad \frac{g' = g[p \mapsto I_p(e, g_p)] \qquad n(p) = \mathbf{run}(e); b \qquad m = n[p \mapsto b]}{\langle n \mid g\rangle \rightsquigarrow^{\mathsf{run}_p} \langle m \mid g'\rangle}$$

$$\textsc{Ncall}\quad \frac{\forall p, n(p) = \mathbf{call}(f); b(p) \qquad \forall p, m(p) = [\![\mathcal{D}(f)]\!]_p \fatsemi b(p)}{\langle n \mid g\rangle \rightsquigarrow^{\mathsf{call}} \langle m \mid g\rangle}$$

$$\textsc{Ncom}\quad \frac{n(p) = \mathbf{send}_q(v); b \qquad n(q) = \mathbf{recv}_p(u); c \qquad m = n[p \mapsto b][q \mapsto c] \qquad g' = g[q \mapsto u(g_q, v(g_p))]}{\langle n \mid g\rangle \rightsquigarrow^{\mathsf{com}_{p,q}} \langle m \mid g'\rangle}$$

$$\textsc{Nbcast}\quad \frac{n(p) = \mathbf{shout}(v); b \qquad f : (q : \mathsf{Role}) \times (q \neq p) \to \{b' \mid n(q) = \mathbf{listen}_p; b'\} \qquad m(p) = b(v(g_p)) \qquad \forall q, (h : q \neq p) \Rightarrow m(q) = (f(q, h)).1\ v(g_p)}{\langle n \mid g\rangle \rightsquigarrow^{\mathsf{bcast}_p} \langle m \mid g\rangle}$$

Figure 6 Operational Semantics of Networks

$$\begin{array}{rcl}
(\cdot \fatsemi \cdot) & : & \mathsf{Proc}_p \to \mathsf{Proc}_p \to \mathsf{Proc}_p \\
(\mathbf{send}_q(v); b) \fatsemi b' & \triangleq & \mathbf{send}_q(v); (b \fatsemi b') \\
(\mathbf{recv}_q(u); b) \fatsemi b' & \triangleq & \mathbf{recv}_q(u); (b \fatsemi b') \\
(\mathbf{listen}_q; b) \fatsemi b' & \triangleq & \mathbf{listen}_q; (\lambda m.\ (b\ m) \fatsemi b') \\
(\mathbf{shout}(v); b) \fatsemi b' & \triangleq & \mathbf{shout}(v); (\lambda m.\ (b\ m) \fatsemi b') \\
(\mathbf{run}(e); b) \fatsemi b' & \triangleq & \mathbf{run}(e); (b \fatsemi b') \\
(\mathbf{call}(f); b) \fatsemi b' & \triangleq & \mathbf{call}(f); (b \fatsemi b') \\
\mathbf{done} \fatsemi b' & \triangleq & b'
\end{array}$$

Figure 7 Sequencing of processes.

value. Also, *every* other process q in the network must be ready to **listen** for a message from p (this is ensured by the function f, which also returns each process's continuation). The transition updates the entire network n such that p and all receiving processes q advance to their respective continuations based on the broadcast value $v(g_p)$. Note that $(f(q, h)).1$ evaluates to the continuation b' of a given process. Finally, in rule Ncall, every process has to be in a state where it wants to call the procedure x next. Then, all processes enter the procedure. Just like in the case of choreographies, we make use of a sequencing operation. It is defined in Figure 7, following the same recursive approach as sequencing for choreographies.

3.5 Correctness Proofs

Now that we have formalized the semantics of choreographies and networks as well as endpoint projection, we want to show that endpoint projection preserves semantics. Furthermore, while arbitrary networks can contain arbitrary deadlocks, we prove that *projected networks* are free from deadlocks, where a network n is a projected network if there exists a choreography c such that $n = [\![c]\!]$.

We proceed by first proving completeness (each choreography step is simulated by a network step) and soundness (each network step is simulated by a choreography step)

of endpoint projection. This ensures the semantics of choreographies and their projected networks match each other. We then prove progress for choreographies (every choreography is either done or can do a step), which implies progress for projected networks, by completeness. Finally, we prove deadlock freedom as follows: Network progress means a projected network n is either done or can perform a step to a network n'. By soundness, we have that n' is itself a projected network, so progress applies again. By induction, we conclude that after an arbitrary number of steps, the network is either done or can perform yet another step. This is the deadlock freedom property.

We begin with completeness. Given choreographies c, c', the completeness theorem states that any step from c to c' corresponds to a step between the network projections $[\![c]\!]$ and $[\![c']\!]$.

▶ **Theorem 1** (Completeness). *For all global states g, g', choreographies c, c', and labels l, if the choreography can step from c to c' with label l and state change g to g', that is, $\langle c \mid g\rangle \rightarrow^l \langle c' \mid g'\rangle$, then the projected network can make a corresponding step $\langle [\![c]\!] \mid g\rangle \rightsquigarrow^l \langle [\![c']\!] \mid g'\rangle$.*

Proof. By induction on the derivation of the step $\langle c \mid g\rangle \rightarrow^l \langle c' \mid g'\rangle$:

- CRUN: When the choreography executes a method at process p, this corresponds directly to a method step NRUN in the network projection at the same process.
- CCOM: Sending a message from process p to q in the choreography corresponds to a sending and receiving step NCOM between the same processes in the network projection.
- CDELAYX: The derivation of a delay step contains either a method step CRUN or communication step CCOM. By the induction hypothesis, such a step can be mapped to a network step.
- CBCAST: Broadcast steps in the choreography correspond to broadcast steps in the network NBCAST after projection.
- CCALL: Call steps in the choreography correspond to call steps in the network NCALL after projection. ◀

In the case for CCALL, applying the NCALL step involves proving that its second precondition is fulfilled, which in this case amounts to showing that, for any role p,

$$[\![\mathcal{D}(x) \fatsemi c]\!]_p = [\![\mathcal{D}(x)]\!]_p \fatsemi [\![c]\!]_p.$$

This is an immediate consequence of the following lemma.

▶ **Lemma 2** (Projection distributes over composition). *For choreographies c, c' and role p, we have $[\![c \fatsemi c']\!]_p = [\![c]\!]_p \fatsemi [\![c']\!]_p$.*

Proof. By induction on c. ◀

Having proven completeness, we now show the converse: The soundness theorem states that any step taken by a projected network corresponds to a valid step in the choreography semantics. While completeness maps choreography steps to network steps, soundness cannot map arbitrary network steps to choreography steps, since the network semantics allows for more general behavior. Hence, we require that the network is the result of projecting a choreography. In the proof, for any network step, we need to inductively traverse this choreography to find the corresponding choreography step.

▶ **Theorem 3** (Soundness). *For any global states g, g', choreography c, network n, and label l, if the network starting from the projection of c can step to n with label l and state change g to g', i.e., $\langle [\![c]\!] \mid g\rangle \rightsquigarrow^l \langle n \mid g'\rangle$, then there exists a choreography c' such that $n = [\![c']\!]$ and the choreography can step correspondingly: $\langle c \mid g\rangle \rightarrow^l \langle c' \mid g'\rangle$.*

Proof. By induction on the structure of the choreography c:

- $\mathsf{run}_p(e); c'$ or $\mathsf{com}_{p,q}(v, u); c'$: We check if the network step matches the operation. If so, we know that the first operation of the choreography is executed, and return the corresponding choreography step (Crun or Ccom, respectively). Otherwise, we apply one of the delay rules to skip the current operation. We then inductively search the rest of the choreography (c') for an operation that matches the network step.
- $\mathsf{bcast}_p(v); c'$: If the first operation is a broadcast, then all projected processes must either have **shout** or **listen** as their first operation. Therefore, the network step can only be a broadcast step Nbcast. We map it to the corresponding choreography step Cbcast.
- $\mathsf{call}(f); c'$: Similarly, if the first operation is a method call, then all projected processes must have the message call as their first operation. Therefore, the network step can only be Ncall, which we map to Ccall.
- done: No step can be done, so the property holds vacuously. ◀

We have shown the correctness of endpoint projection with respect to the operational semantics. We are now left to show progress and deadlock freedom.

The progress lemma for choreographies asserts that any choreography is either finished (done) or can perform a step in the operational semantics.

▶ **Lemma 4** (Choreography Progress). *For any choreography c and global state g:*

$$(c = \mathsf{done}) \vee (\exists c', l, g'.\ \langle c \mid g\rangle \rightarrow^{l} \langle c' \mid g'\rangle)$$

In words: Either the choreography c is already terminated, or there exists a label l, a next choreography c', and an updated global state g', such that $\langle c \mid g\rangle$ can take an l-labeled step to $\langle c' \mid g\rangle$.

Proof. By case distinction on the structure of the choreography: If the choreography has form $\mathsf{com}_{p,q}(u, v, c)$, $\mathsf{bcast}_p(v); c$, $\mathsf{run}_p(e); c$, or $\mathsf{call}(f); c$, there is an obvious corresponding step in the operational semantics that executes the first operation in the choreography. If the choreography is done, the theorem holds trivially. ◀

The progress lemma for the network semantics guarantees that the projection of any choreography either has all processes terminated or can make a network step according to the operational semantics.

▶ **Lemma 5** (Network Progress). *For any global state g and choreography c:*

$$(\forall p.\ [\![c]\!]_p = \mathsf{done}) \vee (\exists c', l, g'.\ \langle [\![c]\!] \mid g\rangle \rightsquigarrow^{l} \langle [\![c']\!] \mid g'\rangle)$$

That is, either every process in the projected network is done, or there exists a network step labeled l from $[\![c]\!]$ at state g to the projected next choreography at the updated global state g'.

Proof. The result follows from progress (Lemma 4) and completeness (Theorem 1) for choreographies:

- By the *progress* lemma (Lemma 4), the choreography c is either done or can make a step from c to c' labeled l from state g to g'.
- If c is done, then by definition all processes in the projected network are also done.
- If c can make a step, then by *completeness* (Theorem 1), the projected network $[\![c]\!]$ can make a corresponding network step.
- Thus, the network semantics either terminates or can make progress in lockstep with the choreography semantics. ◀

The deadlock freedom theorem states that any reachable state of the network semantics, derived from a choreography, is either terminated or can make a further step; hence, no deadlock occurs. For this theorem, we use the relation $\langle n \mid g\rangle \rightsquigarrow^{\star} \langle n' \mid g'\rangle$, which is the reflexive-transitive closure of the network step relation.

▶ **Theorem 6** (Deadlock Freedom). *For any global states g, g', choreography c, and network state n such that the projected choreography $[\![c]\!]$ steps star-wise (zero or more steps) to n at state g':*

$$(\langle [\![c]\!] \mid g\rangle \rightsquigarrow^{\star} \langle n \mid g'\rangle) \Rightarrow (\forall x.\ n\ x = \textbf{done}) \vee (\exists c', l, g'.\ \langle n \mid g\rangle \rightsquigarrow^{l} \langle [\![c']\!] \mid g'\rangle)$$

In words: From any reachable network state n after zero or more steps, either all processes are done or the network can make a further step; thus, the network semantics is deadlock-free.

Proof. We strengthen the conclusion to the following invariant, which we prove to hold by induction over the series of network steps $(\langle [\![c]\!] \mid g\rangle \rightsquigarrow^{\star} \langle n \mid g'\rangle)$:

$$(\exists c.\ ([\![c]\!] = n)) \wedge \big((\forall x.\ n\ x = \textbf{done}) \vee \exists c', l, g'.\ \langle n \mid g\rangle \rightsquigarrow^{l} \langle [\![c']\!] \mid g'\rangle\big)\,.$$

For this, it is necessary to show that (i) a projected choreography satisfies the invariant, and (ii) the invariant is preserved by a network step. Part (i) follows from network progress, while (ii) follows from network progress and soundness. ◀

3.6 Interpreter

In this subsection, we prove partial correctness of a deterministic interpreter for choreographies. More precisely, if the interpreter successfully returns a result, then this result is reachable in the operational semantics. The interpreter we verify is defined as follows:

```
def Chor.interp (d : sig.Func → Chor) (g : GState) : Chor → Nat → Option GState
| run p e c, n+1 => c.interp d g[p ↦ sig.I p e (g p)] n
| com p q _h v u c, n+1 => c.interp d g[q ↦ u (g q) (v (g p))] n
| bcast p v c, n+1 => (c (v (g p))).interp d g n
| call x c, n+1 => (d x).interp d g n >>= (c.interp d . n)
| done, _+1 => g
| _, 0 => .none
```

Because our choreographic language allows nonterminating programs, our interpreter is necessarily non-total. We use a standard *fuel*-based approach [1] to encode this fact in Lean. The interpreter's partiality is indicated by its `Option` return type. It takes a fuel argument of type `Nat` setting a limit on the recursion depth. The fuel argument ensures that the function is structurally recursive, despite the nested recursion in the `call` case, which would not be well-founded otherwise. While we previously defined a simpler interpreter in Section 2.4, the version shown here is more suitable for formal reasoning, as it supports proofs by natural induction.

To begin, we need some lemmas about sequencing. We first prove that sequencing is associative. We will need this as the semantics for procedure calls introduces the sequencing operator into our programs.

▶ **Lemma 7** (Associativity of Sequencing). *For choreographies c, d, e, we have*

$$(c \fatsemi d) \fatsemi e = c \fatsemi (d \fatsemi e).$$

Proof. By induction on c. ◀

Now, we prove two lemmas showing that sequencing is compatible with the operational semantics. The first lemma states that if a reduction step is valid on a choreography, it remains valid when additional code is added to the end of the choreography (using the sequencing operator $\fatsemi$).

▶ **Lemma 8.** *For choreographies c, c', c'', global states g, g', and label l, if $\langle c \mid g\rangle \to^l \langle c' \mid g'\rangle$, then $\langle c \fatsemi c' \mid g\rangle \to^l \langle c' \fatsemi c'' \mid g'\rangle$.*

Proof. By induction on the operational semantics rules. Each rule is essentially mapped to itself, so we just discuss the Crun case, as the others are similar. By assumption, we have $\langle \mathsf{run}_p(e); c' \mid g\rangle \to^l \langle c' \mid g'\rangle$ for some c', g, l, c', g', and want to show $\langle (\mathsf{run}_p(e); c') \fatsemi c'' \mid g\rangle \to^l \langle c' \fatsemi c'' \mid g'\rangle$. By definition of sequencing, this is equal to $\langle \mathsf{run}_p(e); (c' \fatsemi c'') \mid g\rangle \to^l \langle c' \fatsemi c'' \mid g'\rangle$, which is a direct consequence of the Crun rule. In the case of the Ccall rule, we additionally have to use associativity of sequencing. ◀

The second lemma state that this property also holds for the reflexive transitive closure of the semantics.

▶ **Lemma 9.** *For choreographies c, c', global states g, g', and label l, if $\langle c \mid g\rangle \to^\star \langle c' \mid g'\rangle$, then $\langle c \fatsemi c' \mid g\rangle \to^\star \langle c' \fatsemi c'' \mid g'\rangle$.*

Proof. By induction on the derivation of $\langle c \mid g\rangle \to^\star \langle c' \mid g'\rangle$. The base case is trivial, the step case uses Lemma 8. ◀

We can finally prove partial correctness of our interpreter.

▶ **Theorem 10** (Partial Correctness of Interpreter)**.** *For choreography c, global states g, g', and natural number n, if the interpreter given n fuel evaluates c in state g to a state g', then $\langle c \mid g\rangle \to^\star \langle \mathsf{done} \mid g'\rangle$.*

Proof. We want to return $\langle c \mid g\rangle \to^\star \langle \mathsf{done} \mid g'\rangle$. First, we perform induction on n. If $n = 0$, the interpreter cannot return a state, so the statement hold vacuously. If $n = m + 1$, we perform a case distinction on c.

- $\mathsf{run}_p(e); c'$ or $\mathsf{com}_{p,q}(v, u); c'$ or $\mathsf{bcast}_p(v); c'$: The induction hypothesis gives us

  $$\langle c'' \mid g''\rangle \to^\star \langle \mathsf{done} \mid g'\rangle,$$

  where $\langle c'' \mid g''\rangle$ can be reached from $\langle c' \mid g'\rangle$ in one step, by using the rule in the semantics corresponding to the construct (Crun, Ccom, or Cbcast). We compose the respective rule (schematically: $\langle c \mid g\rangle \to^l \langle c'' \mid g''\rangle$) with the induction hypothesis to get $\langle c \mid g\rangle \to^\star \langle \mathsf{done} \mid g'\rangle$.
- $\mathsf{call}(f); c'$: In the interpreter, this case involves nested recursion, so we first perform a case distinction on the inner recursive call. If it fails, then the whole call fails, so the statement hold vacuously. Otherwise, the idea is that reducing $\mathsf{call}(f); c'$ consists of three steps: First, Ccall is applied. Second, $\mathcal{D}f \fatsemi c'$ is reduced. Third, c' is reduced. We get the reduction of $\mathcal{D}f$ via the induction hypothesis, which we then lift with Lemma 9 to a reduction on $\mathcal{D}f \fatsemi c'$. We compose (i) Ccall, (ii) the reduction on $\mathcal{D}f \fatsemi c'$, and (iii) the reduction on c' (which we also get via the induction hypothesis), to get the complete execution.
- done: By reflexivity, we immediately have $\langle \mathsf{done} \mid g\rangle \to^\star \langle \mathsf{done} \mid g\rangle$. ◀

4 Related Work

Choreographies Montesi [17] introduced Choreographic Programming, a more thorough introduction is Montesi [18].

Cruz-Filipe et al. [9] pioneered mechanized certification of choreography compilation in Rocq/Coq.

Hirsch and Garg [12] tackled higher-order typed choreographies in the functional setting with *Pirouette*.

Shen et al. [20] implemented choreographic programming as a Haskell library *HasChor*.

Session Types Session types were originally introduced as *binary* session types between two participants [13]. The session types describe the sequence of messages exchanged by the participants, and ensures that well-typed programs do not deadlock. A later extension are *multiparty* session types [3, 4, 15]. Multiparty session types (MPST) are similarly to choreographies in giving a unified description of a distributed protocol. The main difference is that MPST are, as the name implies, types, and hence separate the specification of a protocol from its implementation. Choreographies, on the other hand, are directly executable programs, but may give less control over the implementation and message-passing-behavior of the protocol. Further, just as choreographies are projected to processes, so global multiparty session types are projected to local types.

Thiemann mechanized deadlock freedom for binary session types using a de Bruijn representation [21], and later introduced the state transformer approach to embedding session types in a functional language [22], focussing on binary session types, and arguing for deadlock freedom without mechanized proofs.

Castro-Perez et al. [6] mechanize a novel approach to multiparty session types where processes are directly checked against the global type, without projection.

Tirore et al. [23] point out flaws in the proof in the original multiparty session types paper [14] and provide a mechanization of soundness for a restricted theory.

Ekici et al. [11] mechanize multiparty session types with subtyping, pointing out unsoundness in several previous papers on the same topic.

5 Conclusion

We have demonstrated the utility of the state transformer approach for mechanization of distributed programming by formalizing a choreographic language with point-to-point communication, broadcasting, recursive procedures, and role-specific local methods. The state transformer approach avoids dealing with variable binding issues and decouples the distributed and the local parts of computation by letting the meta-language handle the latter. We proved soundness and completeness of endpoint projection, as well as deadlock freedom for projected networks.

Limitations and Future Work Our formalization of broadcasting follows involves the entire network. Similarly, recursive procedure calls involve synchronization. While some practical approaches like HasChor [20] use this form of broadcasting, there are others which use the more fine-grained *knowledge of choice* [5] approach, and extending our mechanization to support this concept or the more advanced multiply-located values and enclaves [2] would improve its practical relevance.

Declaration on GenAI tools

We have used generative AI to fix grammatical and stylistic mistakes, to find some of the cited papers, and to help with turning the Lean mechanization code into Latex mathmode notation and with fixing Latex formatting issues.

References

1. Nada Amin and Tiark Rompf. Type soundness proofs with definitional interpreters. In Giuseppe Castagna and Andrew D. Gordon, editors, *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages, POPL 2017, Paris, France, January 18-20, 2017*, pages 666–679. ACM, 2017. doi:10.1145/3009837.3009866.
2. Mako Bates and Joseph P. Near. We know I know you know; choreographic programming with multicast and multiply located values. *CoRR*, abs/2403.05417, 2024. URL: https://doi.org/10.48550/arXiv.2403.05417, arXiv:2403.05417, doi:10.48550/ARXIV.2403.05417.
3. Lorenzo Bettini, Mario Coppo, Loris D'Antoni, Marco De Luca, Mariangiola Dezani-Ciancaglini, and Nobuko Yoshida. Global progress in dynamically interleaved multiparty sessions. In Franck van Breugel and Marsha Chechik, editors, *CONCUR 2008 - Concurrency Theory, 19th International Conference, CONCUR 2008, Toronto, Canada, August 19-22, 2008. Proceedings*, volume 5201 of *Lecture Notes in Computer Science*, pages 418–433. Springer, 2008. doi:10.1007/978-3-540-85361-9_33.
4. Giuseppe Castagna, Mariangiola Dezani-Ciancaglini, and Luca Padovani. On global types and multi-party sessions. In Roberto Bruni and Jürgen Dingel, editors, *Formal Techniques for Distributed Systems - Joint 13th IFIP WG 6.1 International Conference, FMOODS 2011, and 31st IFIP WG 6.1 International Conference, FORTE 2011, Reykjavik, Iceland, June 6-9, 2011. Proceedings*, volume 6722 of *Lecture Notes in Computer Science*, pages 1–28. Springer, 2011. doi:10.1007/978-3-642-21461-5_1.
5. Giuseppe Castagna, Mariangiola Dezani-Ciancaglini, and Luca Padovani. On global types and multi-party session. *Log. Methods Comput. Sci.*, 8(1), 2012. doi:10.2168/LMCS-8(1:24)2012.
6. David Castro-Perez, Francisco Ferreira, and Sung-Shik Jongmans. A synthetic reconstruction of multiparty session types. *Proc. ACM Program. Lang.*, 10(POPL), January 2026. doi:10.1145/3776692.
7. Adam Chlipala. Skipping the binder bureaucracy with mixed embeddings in a semantics course (functional pearl). *Proc. ACM Program. Lang.*, 5(ICFP):1–28, 2021. doi:10.1145/3473599.
8. Luís Cruz-Filipe and Fabrizio Montesi. Choreographies in practice. In Elvira Albert and Ivan Lanese, editors, *Formal Techniques for Distributed Objects, Components, and Systems - 36th IFIP WG 6.1 International Conference, FORTE 2016, Held as Part of the 11th International Federated Conference on Distributed Computing Techniques, DisCoTec 2016, Heraklion, Crete, Greece, June 6-9, 2016, Proceedings*, volume 9688 of *Lecture Notes in Computer Science*, pages 114–123. Springer, 2016. doi:10.1007/978-3-319-39570-8_8.
9. Luís Cruz-Filipe, Fabrizio Montesi, and Marco Peressotti. Certifying choreography compilation. In Antonio Cerone and Peter Csaba Ölveczky, editors, *Theoretical Aspects of Computing - ICTAC 2021 - 18th International Colloquium, Virtual Event, Nur-Sultan, Kazakhstan, September 8-10, 2021, Proceedings*, volume 12819 of *Lecture Notes in Computer Science*, pages 115–133. Springer, 2021. doi:10.1007/978-3-030-85315-0_8.
10. Luís Cruz-Filipe, Fabrizio Montesi, and Marco Peressotti. A formal theory of choreographic programming. *J. Autom. Reason.*, 67(2):21, 2023. URL: https://doi.org/10.1007/s10817-023-09665-3, doi:10.1007/S10817-023-09665-3.
11. Burak Ekici, Tadayoshi Kamegai, and Nobuko Yoshida. Formalising subject reduction and progress for multiparty session processes. In Yannick Forster and Chantal Keller, editors, *16th International Conference on Interactive Theorem Proving, ITP 2025, Reykjavik, Iceland, September 28 - October 1, 2025*, volume 352 of *LIPIcs*, pages 19:1–19:23. Schloss Dagstuhl -

Leibniz-Zentrum für Informatik, 2025. URL: https://doi.org/10.4230/LIPIcs.ITP.2025.19, doi:10.4230/LIPICS.ITP.2025.19.

12 Andrew K. Hirsch and Deepak Garg. Pirouette: higher-order typed functional choreographies. *Proc. ACM Program. Lang.*, 6(POPL):1–27, 2022. doi:10.1145/3498684.

13 Kohei Honda. Types for dyadic interaction. In Eike Best, editor, *CONCUR'93*, pages 509–523, Berlin, Heidelberg, 1993. Springer Berlin Heidelberg.

14 Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. In George C. Necula and Philip Wadler, editors, *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2008, San Francisco, California, USA, January 7-12, 2008*, pages 273–284. ACM, 2008. doi:10.1145/1328438.1328472.

15 Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. *J. ACM*, 63(1), March 2016. doi:10.1145/2827695.

16 Oleg Kiselyov and Hiromi Ishii. Freer monads, more extensible effects. In Ben Lippmeier, editor, *Proceedings of the 8th ACM SIGPLAN Symposium on Haskell, Haskell 2015, Vancouver, BC, Canada, September 3-4, 2015*, pages 94–105. ACM, 2015. doi:10.1145/2804302.2804319.

17 Fabrizio Montesi. *Choreographic Programming*. PhD thesis, IT University of Copenhagen, 2013. https://www.fabriziomontesi.com/files/choreographic-programming.pdf.

18 Fabrizio Montesi. *Introduction to Choreographies*. Cambridge University Press, 2023. URL: https://www.cambridge.org/core/books/introduction-to-choreographies/65D3DA3CFF11AB835452CBC97FAE4830, doi:10.1017/9781108981491.

19 Marco Patrignani. Why should anyone use colours? or, syntax highlighting beyond code snippets. *CoRR*, abs/2001.11334, 2020. URL: https://arxiv.org/abs/2001.11334, arXiv:2001.11334.

20 Gan Shen, Shun Kashiwa, and Lindsey Kuper. Haschor: Functional choreographic programming for all (functional pearl). *Proc. ACM Program. Lang.*, 7(ICFP):541–565, 2023. doi:10.1145/3607849.

21 Peter Thiemann. Intrinsically-typed mechanized semantics for session types. In *Proceedings of the 21st International Symposium on Principles and Practice of Declarative Programming*, PPDP '19, New York, NY, USA, 2019. Association for Computing Machinery. doi:10.1145/3354166.3354184.

22 Peter Thiemann. Intrinsically typed sessions with callbacks (functional pearl). *Proc. ACM Program. Lang.*, 7(ICFP):711–739, 2023. doi:10.1145/3607854.

23 Dawit Legesse Tirore, Jesper Bengtson, and Marco Carbone. Multiparty asynchronous session types: A mechanised proof of subject reduction. In Jonathan Aldrich and Alexandra Silva, editors, *39th European Conference on Object-Oriented Programming, ECOOP 2025, Bergen, Norway, June 30 - July 2, 2025*, volume 333 of *LIPIcs*, pages 31:1–31:30. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025. URL: https://doi.org/10.4230/LIPIcs.ECOOP.2025.31, doi:10.4230/LIPICS.ECOOP.2025.31.

24 Li-yao Xia, Yannick Zakowski, Paul He, Chung-Kil Hur, Gregory Malecha, Benjamin C. Pierce, and Steve Zdancewic. Interaction trees: representing recursive and impure programs in coq. *Proc. ACM Program. Lang.*, 4(POPL):51:1–51:32, 2020. doi:10.1145/3371119.